



Rapport de Projet : MLP et moteur d'autodiff appliqués à MNIST

Auteurs:

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

Encadrant:

Aurélien Delval

Responsables:

Aurélien Delval / Hugo Bolloré / Pablo Oliveira

December 19, 2025

Sommaire

1	Introduction	5
2	Guide de lecture	5
3	Contexte et État de l’art	6
3.1	Introduction au problème	6
3.2	Concept de la méthode classique	6
3.3	Présentation des éléments modulables du MLP	8
3.3.1	Fonctions d’activation	8
3.3.2	Algorithmes de descente de gradient	8
3.4	Travaux existants et évolution des architectures	9
3.5	Synthèse : du modèle naïf au réseau moderne	9
3.6	Périmètre du projet	9
4	Implémentation	10
4.1	Choix techniques et justification	10
4.1.1	Implémentation	10
4.1.2	Matrix / Node / graphe de calcul	10
4.1.3	Architecture générale	10
4.2	Architecture du MLP et organisation du code	10
4.2.1	Couche de base : opérations numériques et graphe computationnel	10
4.2.2	Architecture du réseau	11
4.2.3	Module d’entraînement	11
4.3	Implémentation détaillée	11
4.3.1	Implémentation de Matrix	12
4.3.2	Construction du graphe computationnel (Node , MathOps)	13
4.3.3	Implémentation des couches du MLP	14
4.3.4	Implémentation de la fonction de <i>loss</i> et de l’optimisation	14
4.3.5	Boucle d’entraînement	15
5	Expérimentations	16
5.1	Configuration de l’environnement	16
5.1.1	Environnement Matériel / Hardware	16
5.1.2	Environnement Logiciel / Software	16
5.1.3	Configuration Spécifique et Protocole	16
5.2	Analyse des performances de calcul et d’entraînement	17
5.2.1	Passage à l’échelle (Thread Scaling)	17
5.2.2	Hiérarchie des performances et Speedup	19
5.2.3	Analyse de Profiling et Loi d’Amdahl : vers l’entraînement complet	20
5.3	Convergence de l’entraînement	21
5.3.1	Premiers tests de convergence	21
5.3.2	Impact du Taux d’Apprentissage (Learning Rate)	22
5.3.3	Impact de la Taille de Batch (Batch Size)	24
5.3.4	Impact de la Taille de la Couche Cachée (Hidden Size)	25
5.3.5	Impact de la Fonction d’Activation et de l’Initialisation	26
5.3.6	Impact de l’Affinité sur les charges d’entrée (Input Layer)	27
5.4	Synthèse des Résultats Expérimentaux	28
6	Déroulé du Projet	28
7	Conclusion & perspectives	28

8	Références	30
9	Glossaire	30
A	Exemple : graphe de calcul et autodiff	32
B	Protocole de Verrouillage des Fréquences	32
C	Scripts d'Expérimentation	32

List of Figures

1	Architecture d'un Perceptron Multicouche (MLP) avec couches cachées [3].	7
2	Diagramme de classes UML (implémentation détaillée)	12
3	Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices.	17
4	Speedup (accélération) en fonction du nombre de threads.	18
5	Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS).	19
6	Speedup relatif des différentes optimisations.	19
7	Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS.	21
8	Courbe de <i>loss</i> et de précision (<i>accuracy</i>) sur les données d'entraînement (60 000 images) et de validation (10 000 images).	22
9	Impact du <i>learning rate</i> sur la convergence (10 époques, 3 graines). Les axes de précision utilisent une échelle discontinue pour visualiser simultanément les hautes précisions ($> 90\%$) et les échecs de convergence ($< 40\%$).	23
10	Impact de la Taille de Batch (20 époques, $LR = 0.01$, 3 graines). Les petits batches (16, 32) convergent mieux mais sont plus lents à exécuter. Le batch 256 diverge avec ce LR fixe.	24
11	Impact de la Taille de la Couche Cachée (20 époques, $LR = 0.01$, $B = 64$, 3 graines). L'augmentation de la taille améliore la précision, mais avec des rendements décroissants au-delà de 128 neurones.	25
12	Comparaison des Fonctions d'Activation et Initialisation. ReLU (He) et ReLU (Manual) offrent les meilleures performances. Sigmoid et Tanh (avec Xavier) restent compétitifs grâce à une initialisation adaptée.	26

List of Tables

1	Configuration matérielle	16
2	Configuration logicielle	16
3	Profil d'exécution (gprof) - Version Naïve : Matrix::matmul domine (96.4%)	20
4	Profil d'exécution (gprof) - Version BLAS : matmul négligeable, overhead mémoire dominant	21
5	Configuration standard pour le test de convergence.	22
6	Impact de l'affinité sur les performances pour une multiplication matricielle $64 \times 784 \times 128$	27

1 Introduction

Ce document présente le rapport du projet de construction d'un réseau de neurones multicouche (MLP) et d'un moteur de différenciation automatique appliqué au jeu de données MNIST [2]. Réalisé dans le cadre de l'UE Projet en première année de master de calcul haute performance, simulation à l'Université Paris Saclay, ce travail a pour objectif d'implémenter pas à pas les fondements d'un système d'apprentissage automatique sans dépendre de bibliothèques haut niveau.

L'objectif général du projet est double :

1. Comprendre et implémenter les mécanismes internes d'un réseau neuronal, incluant le passage avant, la rétropropagation et la mise à jour de paramètres.
2. Construire un moteur d'autodiff permettant de dériver automatiquement les expressions mathématiques du réseau, afin de calculer les gradients nécessaires à l'apprentissage.

En parallèle, le projet vise à familiariser les étudiants avec l'analyse expérimentale, l'optimisation du code et la préparation aux travaux de performance qui seront approfondis au second semestre.

2 Guide de lecture

Le rapport est structuré de manière à accompagner progressivement le lecteur. Il s'articule autour des parties suivantes :

Partie 1 : Contexte et État de l'art : présentation du problème MNIST et du Perceptron Multicouche.

Partie 2 : Implémentation : description détaillée de l'architecture logicielle, du moteur d'autodifférentiation et des choix techniques.

Partie 3 : Expérimentations : double analyse portant sur les performances de calcul (HPC, profiling) et la validation de la convergence du modèle (Machine Learning).

Partie 4 : Déroulé du projet : présentation de l'organisation du travail et de la répartition des tâches au sein de l'équipe.

Enfin, une **Conclusion** dresse le bilan et esquisse les perspectives, suivie des **Annexes** techniques, des références et d'un glossaire.

3 Contexte et État de l'art

3.1 Introduction au problème

Au cours des dernières décennies, l'apprentissage automatique a connu un développement considérable, principalement grâce aux progrès réalisés dans le domaine des réseaux de neurones artificiels. Parmi les problèmes emblématiques de ce domaine figure la reconnaissance de chiffres manuscrits, couramment étudiée à l'aide du jeu de données MNIST [2], devenu un standard pour les tâches de classification. Ce travail vise à présenter les approches classiques permettant de résoudre ce problème, en commençant par le perceptron multicouche (MLP), avant d'aborder ses principales variantes et l'évolution des modèles plus récents.

3.2 Concept de la méthode classique

Le problème MNIST consiste à classer des images de chiffres manuscrits (28×28 pixels) dans dix catégories correspondant aux chiffres 0 à 9. Pour résoudre ce problème, on utilise un perceptron multicouche (MLP), c'est-à-dire un réseau de neurones constitué de plusieurs neurones artificiels organisés en couches (entrée, cachées, sortie).

Chaque neurone calcule une combinaison linéaire de ses entrées, à laquelle il applique ensuite une fonction d'activation non linéaire. Soit x le vecteur d'entrée, w le vecteur de poids et b le biais. La sortie y d'un neurone est donnée par :

$$y = \sigma(w \cdot x + b)$$

Passage au calcul matriciel On commence par le cas d'un seul exemple. Une image MNIST de taille 28×28 est vectorisée (*flattened*) en un vecteur d'entrée $x \in \mathbb{R}^{1 \times 784}$. Pour une couche entièrement connectée comportant n neurones, on introduit une matrice de poids $W \in \mathbb{R}^{784 \times n}$ et un biais $b \in \mathbb{R}^{1 \times n}$. La pré-activation s'écrit alors :

$$z = xW + b$$

où $z \in \mathbb{R}^{1 \times n}$. Cette transformation correspond à une application affine de l'espace des pixels (dimension 784) vers un espace de représentation de dimension n , avant l'application de la non-linéarité σ .

Dans la pratique, un traitement séquentiel (image par image) exploite imparfaitement les ressources matérielles. Afin d'augmenter le débit (*throughput*), on recourt au calcul par *mini-batches* (*mini-batches*), qui permet de reformuler une suite de produits vecteur-matrice en un unique produit matrice-matrice, particulièrement optimisé (SIMD, multi-cœurs, BLAS, et, le cas échéant, GPU).

Plus précisément, en regroupant N exemples, on définit la matrice d'entrées $X \in \mathbb{R}^{N \times 784}$. La propagation avant se généralise en :

$$Y = \sigma(XW + \mathbf{1}_N b)$$

où $\mathbf{1}_N \in \mathbb{R}^{N \times 1}$ désigne un vecteur de 1 et $\mathbf{1}_N b$ formalise la diffusion (*broadcast*) du biais sur l'ensemble des N exemples.

Pour clarifier ce mécanisme, la figure suivante illustre d'abord le cas xW , puis son extension naturelle au cas matriciel avec XW .

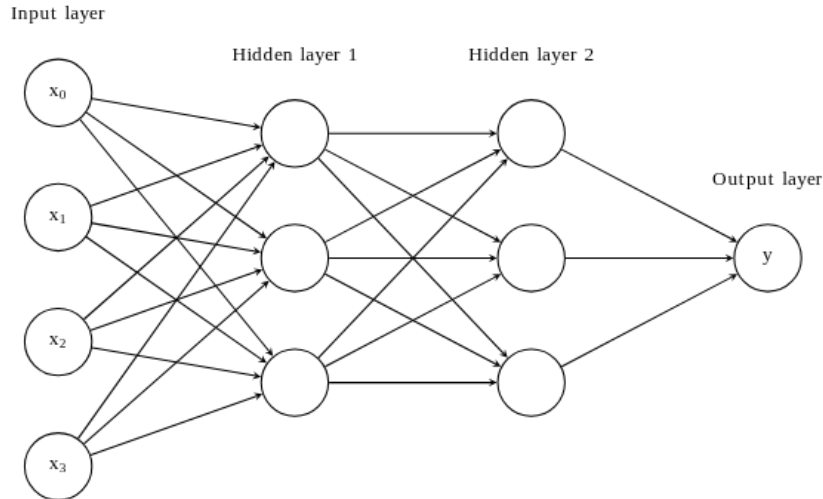


Figure 1: Architecture d'un Perceptron Multicouche (MLP) avec couches cachées [3].

Cette figure illustre la structure hiérarchique du modèle (schéma purement illustratif : pour MNIST, chaque entrée est un vecteur de dimension 784, et un mini-batch correspond à une matrice X de dimensions $N \times 784$) :

Couche d'entrée (x_0 à x_3) : reçoit les données brutes (ici les pixels d'une image).

Couches cachées : couches intermédiaires où le réseau apprend des représentations abstraites ; chaque neurone est connecté à tous les neurones de la couche précédente (*fully connected*).

Couche de sortie (y) : produit la prédiction finale (probabilités des classes pour MNIST).

Bien que le schéma montre des connexions individuelles, notre implémentation vectorise ces opérations sous forme de produits matriciels $X \cdot W$. C'est cette opération matricielle que nous optimisons intensivement dans ce projet (BLAS, OpenMP).

L'objectif de l'apprentissage est de minimiser une fonction de *loss* (*loss*) L , qui quantifie l'écart entre les prédictions du modèle et les étiquettes réelles. Minimiser L revient donc à réduire cet écart sur les données d'entraînement (et, idéalement, à améliorer la capacité de généralisation sur des exemples non vus). Dans la suite, la rétropropagation (*backpropagation*) [4] permet de calculer le gradient ∇L par rapport à tous les paramètres afin d'effectuer cette minimisation.

Règle de la chaîne (*chain rule*) Le réseau peut se voir comme une composition de fonctions $f(x) = f_n(f_{n-1}(\dots f_1(x)))$. Le gradient se calcule en remontant de la fin vers le début via la règle de la chaîne :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Cela conduit naturellement à une représentation sous forme de **graphe de calcul** (DAG), où chaque nœud est une opération élémentaire, connaissant ses parents et sa dérivée locale. Lors de la phase *backward*, nous propageons le gradient accumulé depuis la perte jusqu'aux feuilles (les poids). Dans le cas général, exprimer analytiquement le gradient d'un réseau profond est difficile et source d'erreurs. L'autodiff permet de le calculer automatiquement en appliquant systématiquement la règle de la chaîne, sans dériver explicitement les formules pour chaque architecture (p. ex. PyTorch, TensorFlow).

Exemple : graphe de calcul et autodiff Un exemple détaillé est donné en Annexe A.

3.3 Présentation des éléments modulables du MLP

Après avoir décrit le fonctionnement général du MLP et les principes du calcul de gradient, nous présentons maintenant les différents éléments modulables de cette architecture.

3.3.1 Fonctions d'activation

Les fonctions d'activation introduisent la non-linéarité dans le réseau. Elles sont cruciales pour deux raisons :

1. **Non-linéarité** : Sans elles, une superposition de couches linéaires resterait une simple transformation linéaire unique ($W_2(W_1x) = W'x$), incapable de modéliser des problèmes complexes (comme XOR ou MNIST).
2. **Recadrage des valeurs** : Elles permettent souvent de maintenir les activations dans une plage bornée (p. ex. $[0,1]$ pour la sigmoïde), évitant l'explosion des valeurs lors de la propagation dans des réseaux profonds.

Sigmoïde : transforme l'entrée en une valeur entre 0 et 1, mais peut provoquer la saturation des gradients.

Tanh : centrée sur zéro, elle améliore souvent la convergence mais reste sujette au même problème de saturation.

ReLU (*Rectified Linear Unit*) : ne conserve que les valeurs positives ($ReLU(x) = \max(0, x)$) ; elle accélère l'apprentissage et constitue aujourd'hui un choix standard [1].

3.3.2 Algorithmes de descente de gradient

L'apprentissage consiste à minimiser la *loss* grâce à la descente de gradient. L'algorithme met à jour les paramètres θ (poids et biais) itérativement :

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L$$

où η est le **taux d'apprentissage** (*learning rate*).

- Un η trop grand accélère la convergence mais risque de provoquer des oscillations, voire une divergence.
- Un η trop petit garantit la stabilité mais ralentit considérablement l'apprentissage.

Listing 1: Algorithme simplifié (SGD)

```
Pour chaque epoch:
  Pour chaque mini-batch (X, Y):
    1. Forward: P = Model(X)
    2. Loss: L = LossFn(P, Y)
    3. Backward: Calculer les gradients via autodiff
    4. Update: W = W - learning_rate * dL/dW
```

Si ce projet se concentre sur la méthode **SGD**, il convient de citer les évolutions majeures de l'état de l'art :

- **SGD** (*Stochastic Gradient Descent*) : met à jour les poids après chaque *mini-batch* (**méthode implémentée**).
- **Momentum** : ajoute une inertie pour lisser les oscillations et accélérer la convergence.
- **Adam** : combine Momentum et adaptation du taux d'apprentissage ; optimiseurs standards actuels.

3.4 Travaux existants et évolution des architectures

Premières approches : MLP et réseaux simples Les premières tentatives d'utilisation de réseaux de neurones pour la reconnaissance des chiffres remontent aux années 1990. Appliqués au jeu de données MNIST, ces modèles atteignaient déjà des taux de réussite proches de 97 à 98 %. Cependant, leur performance restait limitée car ils ne tiraient pas parti de la structure spatiale des images et manquaient d'invariance aux translations.

Réseaux convolutionnels (CNN) Une avancée majeure est survenue avec l'introduction des réseaux de neurones convolutionnels (CNN), proposés notamment par LeCun et al. avec le modèle LeNet-5 [2]. Cette architecture repose sur le principe de convolution locale, où chaque neurone est connecté uniquement à une petite région de l'image. LeNet-5 a atteint un taux de reconnaissance d'environ 99,2 % sur MNIST, établissant un nouveau standard à l'époque.

3.5 Synthèse : du modèle naïf au réseau moderne

L'évolution des architectures appliquées à MNIST illustre parfaitement la progression du domaine de l'apprentissage profond. Néanmoins, la compréhension des fondements du MLP demeure indispensable : elle permet de saisir les principes élémentaires du calcul du gradient, de la propagation de l'erreur et de la représentation hiérarchique des données. Ainsi, notre travail s'inscrit dans cette continuité : commencer par un modèle simple (MLP), en explorer les variantes, avant d'envisager des architectures plus complexes.

3.6 Périmètre du projet

Le projet consiste à concevoir et implémenter un pipeline complet d'apprentissage supervisé comprenant :

- un module de manipulation de tenseurs,
- un réseau de couches linéaires avec fonctions d'activation,
- un moteur de différenciation automatique (autodiff),
- un mécanisme d'entraînement (optimiseur, *loss*, gestion des *mini-batches*),
- l'intégration du jeu de données MNIST et l'évaluation du modèle.

Ce développement a été réalisé en plusieurs phases successives décrites ci-dessous.

4 Implémentation

4.1 Choix techniques et justification

4.1.1 Implémentation

Choix du langage C++ pour sa performance et son contrôle fin de la mémoire, en cohérence avec l'orientation *calcul haute performance* de l'UE. Implémentation d'un moteur de calcul et d'autodifférentiation interne (`Matrix`, `Node`, `MathOps`) afin de comprendre explicitement la construction du graphe computationnel et le calcul des gradients [1], plutôt que de s'appuyer sur des bibliothèques de haut niveau.

4.1.2 Matrix / Node / graphe de calcul

- Représentation des données numériques par une classe `Matrix` générique (doubles en 2D), offrant les opérations de base nécessaires au MLP (`matmul`, `add`, `mul`, `transpose`).
- Représentation du graphe computationnel par la classe `Node`, qui stocke la valeur courante, le gradient associé, les parents et une fonction de rétropropagation (`backwardFn_`).
- Utilisation d'un tri topologique (méthode statique `topoSort`) pour parcourir le graphe lors de la phase de rétropropagation, plutôt que d'utiliser une récursion profonde.

4.1.3 Architecture générale

Choix d'un perceptron multicouche (MLP) à base de couches linéaires plutôt que de réseaux convolutifs ou récurrents, afin de se concentrer sur les mécanismes d'autodiff et de rétropropagation. Introduction d'une interface `ActivationFunction` (implémentée par `ReLU`, `Sigmoid`, `Tanh`) pour permettre de changer de fonction d'activation sans modifier la structure globale du code. Encapsulation de la combinaison « couche linéaire + activation » dans la structure `LayerNode`, puis construction du modèle complet via un vecteur de `LayerNode` dans `MLPNetwork`.

4.2 Architecture du MLP et organisation du code

Cette section décrit l'architecture logicielle du projet ainsi que les relations entre les différents modules, telles que représentées dans le diagramme UML (Figure 2). L'objectif est de mettre en évidence la séparation des responsabilités, l'organisation hiérarchique des composants et le flux global des données au sein du système.

4.2.1 Couche de base : opérations numériques et graphe computationnel

La couche la plus fondamentale de l'architecture repose sur les classes `Matrix`, `Node`, `OperationNode` et `MathOps`. Elle constitue le socle sur lequel s'appuient toutes les composantes du réseau neuronal.

La classe `Matrix` fournit les structures de données nécessaires aux calculs numériques et implémente les opérations linéaires de base utilisées dans le réseau. La classe `Node` modélise les nœuds du graphe computationnel. Chaque nœud contient une valeur courante, un gradient associé, ainsi que des liens vers ses nœuds parents, ce qui permet de représenter explicitement les dépendances entre les opérations. La classe `OperationNode` étend `Node` afin d'identifier le type d'opération réalisée via un identifiant (`OpKind`), facilitant ainsi la lecture et l'analyse du graphe.

Les opérations mathématiques sont centralisées dans la classe `MathOps`, qui encapsule des fonctions telles que `matmul`, `add`, `mul`, `relu`, `sigmoid` ou `tanh`. Chaque appel à `MathOps` construit un nouveau `Node`, définit ses dépendances et associe la fonction de rétropropagation correspondante. Cette organisation permet de découpler le moteur d'autodifférentiation de la définition du réseau, rendant le graphe computationnel générique et réutilisable.

4.2.2 Architecture du réseau

Au-dessus du moteur de calcul, l'architecture du réseau neuronal est construite à l'aide de modules de plus haut niveau, spécifiquement dédiés à la définition du modèle. La classe `LinearLayer` encapsule les paramètres entraînables du réseau, à savoir les poids et les biais, ainsi que l'opération de projection linéaire appliquée aux entrées.

Les fonctions d'activation sont modélisées via l'interface abstraite `ActivationFunction`, permettant une implémentation interchangeable des activations telles que ReLU, Sigmoid ou Tanh. L'unité fonctionnelle de base du réseau est la structure `LayerNode`, qui combine une couche linéaire et une fonction d'activation en un bloc cohérent. Le modèle complet est représenté par la classe `MLPNetwork`, structurée comme une séquence de `LayerNode`, ce qui permet de composer facilement des réseaux multi-couches de profondeur variable. Cette conception favorise la modularité et permet de modifier l'architecture du réseau sans impacter le moteur de calcul sous-jacent.

4.2.3 Module d'entraînement

Le module d'entraînement regroupe l'ensemble des composants nécessaires à l'apprentissage du modèle et à son évaluation. L'abstraction `LossFunction` permet de définir différentes fonctions de *loss*, dont `MSELoss` et `CrossEntropyLoss`, utilisées respectivement pour des tests simples et pour la classification sur MNIST. La mise à jour des paramètres est assurée par la classe abstraite `Optimizer`, avec une implémentation basée sur la descente de gradient stochastique (`SGDOptimizer`).

Les données sont fournies par `MNISTDataset`, puis organisées en mini-batches à l'aide de `DataLoader`. La classe `Trainer` coordonne l'ensemble du processus d'entraînement, en orchestrant le flux suivant : chargement des données, propagation avant dans le modèle, calcul de la *loss*, rétropropagation des gradients et mise à jour des paramètres. Cette séparation claire des rôles permet de faire évoluer indépendamment le modèle, la fonction de *loss* ou la stratégie d'optimisation, tout en conservant une architecture globale cohérente.

4.3 Implémentation détaillée

Cette section décrit les principaux choix d'implémentation des classes et méthodes constituant le moteur de calcul, le réseau neuronal et le pipeline d'entraînement. Après avoir présenté l'architecture globale du projet, nous détaillons ici la manière dont elle se traduit concrètement dans le code. Le diagramme UML de la Figure 2 synthétise les principales classes et relations ; l'implémentation décrite ci-dessous suit cette organisation.

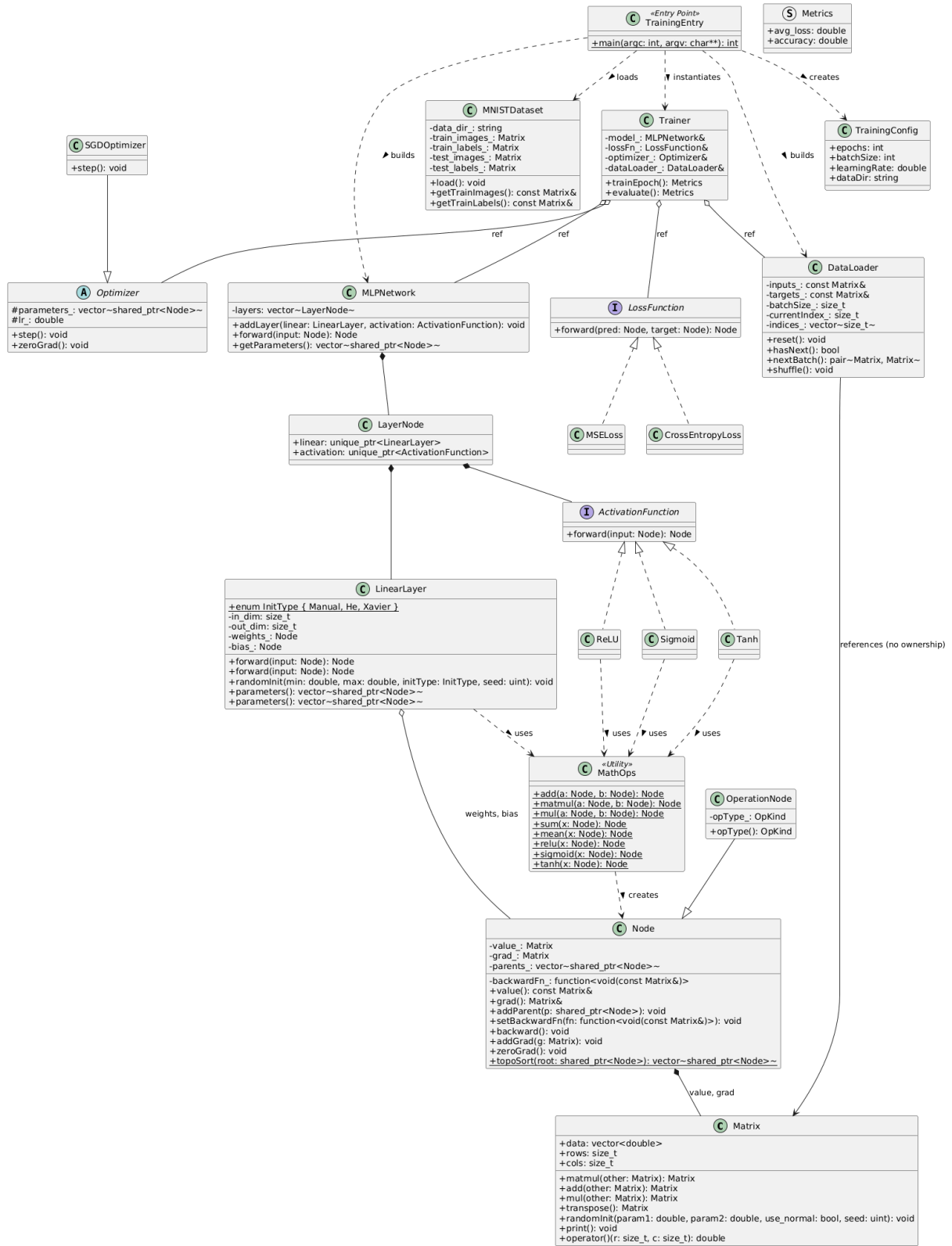


Figure 2: Diagramme de classes UML (implémentation détaillée)

4.3.1 Implémentation de Matrix

La classe `Matrix` constitue la brique de base du calcul numérique. Les données sont encapsulées dans un `std::vector<double>`, garantissant une allocation mémoire contiguë. Ce choix est critique pour la performance : il maximise la localité spatiale et l'efficacité des caches CPU

(L1/L2), un prérequis favorable aux optimisations vectorielles (SIMD).

Listing 2: Structure de données contiguë (Matrix)

```
class Matrix {
public:
    // Stockage contigu garantissant la localité spatiale
    std::vector<double> data;
    size_t rows;
    size_t cols;
    // ...
};
```

Les opérations fondamentales nécessaires au MLP sont implémentées explicitement. Concernant la multiplication matricielle (**matmul**), le code adopte une architecture flexible permettant de sélectionner l’implémentation sous-jacente (via une variable d’environnement) :

- une version naïve (triple boucle i - j - k) servant de référence pédagogique ;
- des versions optimisées (BLAS [7], Blocked, OpenMP [6]) assurant la performance.

Les opérations élémentaires **add**, **mul** et la transposition sont également disponibles.

Ce choix d’implémentation permet de comparer différentes approches d’optimisation (telles que détaillées dans la section expérimentale) tout en conservant une maîtrise totale de la chaîne de calcul.

Concernant l’initialisation des paramètres, nous avons enrichi la méthode **randomInit** pour supporter des stratégies avancées cruciales pour la convergence des réseaux profonds :

- **He (Kaiming) Initialization** : ajustée pour les activations ReLU, utilisant une variance de $2/n_{in}$.
- **Xavier (Glorot) Initialization** : adaptée aux activations Sigmoid/Tanh.

De plus, l’initialisation intègre une gestion explicite de la graine aléatoire (*seed*), garantissant la reproductibilité exacte des poids initiaux entre deux exécutions.

4.3.2 Construction du graphe computationnel (Node, MathOps)

Le moteur d’autodifférentiation repose sur la classe **Node**, dont la gestion mémoire est assurée par des pointeurs intelligents **std::shared_ptr**. Cette approche automatise la gestion du cycle de vie des nœuds au sein du Graphe Acyclique Dirigé (DAG), évitant ainsi les fuites de mémoire complexes inhérentes aux graphes dynamiques.

Chaque **Node** contient une valeur, un gradient, une liste de parents, et surtout une fonction de rétropropagation **backwardFn**, typée via **std::function**. Lorsqu’une opération est invoquée via **MathOps**, l’utilisation d’expressions lambda (*C++ moderne*) permet de capturer efficacement le contexte local (clôtures) nécessaire au calcul du gradient, offrant une syntaxe concise et expressive. Certaines opérations (comme **add**) supportent une diffusion (*broadcasting*) simple ($1 \times M$ vers $N \times M$), notamment pour l’ajout du biais.

Listing 3: Utilisation de lambdas et de pointeurs intelligents (MathOps)

```
// Exemple pour l'opération Mul (e.g. y = a * b)
Node::Ptr mul(const Node::Ptr& a, const Node::Ptr& b) {
    auto node = std::make_shared<OperationNode>(...);

    // Capture du contexte par valeur (pointeurs intelligents)
    node->setBackwardFn([a, b](const Matrix& grad){
        // Calcul local du gradient : dL/da = grad * b
        a->addGrad(grad.mul(b->value()));
        b->addGrad(grad.mul(a->value()));
    });
}
```

```

    });
    return node;
}

```

La phase de rétropropagation s'effectue en trois étapes :

1. un tri topologique du graphe computationnel ;
2. l'initialisation du gradient au niveau de la fonction de *loss* (explicitement, ou par défaut à 1 s'il n'est pas défini) ;
3. l'appel successif des fonctions `backwardFn_` de chaque nœud dans l'ordre inverse du passage avant.

Cette approche garantit une propagation correcte des gradients, tout en évitant les récursions profondes.

4.3.3 Implémentation des couches du MLP

LinearLayer La classe `LinearLayer` implémente la transformation affine du réseau. La méthode `forward(input)` réalise l'opération $\text{output} = \text{input} \times \text{weights} + \text{bias}$ à l'aide des opérateurs définis dans `MathOps`. Les paramètres sont initialisés via la méthode `randomInit()`, et la méthode `parameters()` permet d'exposer les nœuds entraînaibles au module d'optimisation.

Fonctions d'activation Les fonctions d'activation sont implémentées via l'héritage et le polymorphisme d'exécution (*polymorphisme à l'exécution*). Une classe abstraite de base définit l'interface virtuelle pure `forward()`, permettant d'interchanger dynamiquement les algorithmes (ReLU, Sigmoid, Tanh) sans impacter le reste du réseau.

Listing 4: Polymorphisme pour les activations

```

class ActivationFunction {
public:
    // Interface virtuelle pure
    virtual Node::Ptr forward(const Node::Ptr& input) const = 0;
    virtual ~ActivationFunction() = default;
};

class ReLU : public ActivationFunction {
    /* Implémentation concrète */
};

```

LayerNode & MLPNetwork Un `LayerNode` combine une couche linéaire et une fonction d'activation en une unité cohérente. La classe `MLPNetwork` applique successivement ces blocs lors de la propagation avant, permettant de construire des réseaux multi-couches de manière flexible.

4.3.4 Implémentation de la fonction de *loss* et de l'optimisation

L'apprentissage du réseau repose sur l'association d'une fonction de *loss* et d'un algorithme d'optimisation, tous deux intégrés au moteur d'autodifférentiation. Les fonctions de *loss* sont définies via une interface abstraite `LossFunction`, qui impose la méthode `forward(pred, target)`. Celle-ci construit un nœud scalaire représentant la *loss* du mini-batch, directement inséré dans le graphe de calcul.

Deux fonctions de *loss* ont été implémentées :

1. **MSELoss** : utilisée principalement pour les tests et la validation du moteur d'autodifférentiation. Elle correspond à l'erreur quadratique moyenne entre prédictions et cibles, la *loss* retournée étant la somme sur le mini-batch, normalisée par le nombre de sorties.
2. **CrossEntropyLoss** : utilisée pour la classification MNIST. Elle opère directement sur les *logits* du réseau et intègre une opération de *softmax* stable numériquement. Le gradient par rapport aux *logits* s'exprime simplement comme la différence entre les probabilités prédites et les labels *one-hot*, ce qui garantit une rétropropagation efficace et stable.

L'optimisation des paramètres est assurée par une interface abstraite **Optimizer**, tirant parti, là encore, du mécanisme de fonctions virtuelles du C++. La méthode `step()` est redéfinie par les classes filles (comme **SGDOptimizer**), ce qui offre une extensibilité immédiate pour l'ajout futur d'algorithmes plus complexes (Adam, RMSProp) sans modifier la boucle d'entraînement.

4.3.5 Boucle d'entraînement

La boucle d'entraînement et d'évaluation est implémentée dans la classe **Trainer**. Elle repose sur une fonction centrale `runEpoch`, paramétrée par un indicateur permettant de distinguer les phases d'apprentissage et d'évaluation.

Au début de chaque époque, le **DataLoader** est réinitialisé afin de parcourir l'ensemble des mini-batches. Pour chaque mini-batch, les matrices d'entrée et de cibles sont encapsulées dans des nœuds constants du graphe computationnel. La propagation avant est ensuite réalisée en appliquant successivement le modèle (**MLPNetwork**) aux données d'entrée, produisant les prédictions du réseau.

La fonction de *loss* est alors évaluée à partir des prédictions et des cibles, générant un nœud de *loss* généralement scalaire. La valeur numérique de la *loss* est accumulée afin de calculer une *loss* moyenne sur l'ensemble de l'époque. En parallèle, la précision (*accuracy*) est estimée au niveau du **Trainer** en comparant, pour chaque échantillon du mini-batch, la classe prédite (*argmax* des *logits*) à la classe cible encodée en *one-hot*.

Lorsque l'époque est exécutée en mode entraînement, la phase de rétropropagation est déclenchée pour chaque mini-batch. Les gradients des paramètres sont d'abord réinitialisés, puis la méthode `backward` est appelée sur le nœud de *loss* afin de propager les gradients dans le graphe computationnel. Les paramètres du réseau sont enfin mis à jour à l'aide de l'optimiseur. En mode évaluation, les étapes de rétropropagation et de mise à jour des paramètres sont désactivées, ce qui permet de mesurer les performances du modèle sans modifier son état interne.

À la fin de l'époque, la *loss* moyenne et la précision globale sont calculées à partir des quantités accumulées et retournées sous forme de métriques.

5 Expérimentations

5.1 Configuration de l’environnement

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

5.1.1 Environnement Matériel / Hardware

Table 1: Configuration matérielle

Composant	Caractéristiques
Processeur	AMD Ryzen 9 8940HX with Radeon Graphics Architecture : x86_64 Cœurs / Threads : 16 physiques / 32 logiques (2 threads/cœur) Fréquence : 421.8 MHz (min) à 5386 MHz (max) Cache L1d : 512 KiB total, répartis en 16 instances Cache L1i : 512 KiB total, répartis en 16 instances Cache L2 : 16 MiB total, répartis en 16 instances Cache L3 : 64 MiB total, répartis en 2 instances
Mémoire	30 GiB de RAM système
Swap	8.0 GiB

Les expérimentations parallèles ont été réalisées sur cette même configuration matérielle.

5.1.2 Environnement Logiciel / Software

Table 2: Configuration logicielle

Composant	Version / Détails
Système d’exploitation	Fedora Linux 42 (Workstation Edition)
Compilateur C/C++	GCC 15.2.1 20250808 (Red Hat 15.2.1-1)

Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l’auto-vectorisation et les optimisations spécifiques à l’architecture.

5.1.3 Configuration Spécifique et Protocole

Afin de limiter la variabilité des résultats, les campagnes de mesure suivent un protocole systématique de verrouillage de fréquence et d’affinité CPU. Les huit premiers cœurs physiques sont forcés à 4.0 GHz à l’aide de `cpupower`, puis les exécutions sont liées explicitement à ce jeu de cœurs via `taskset`. Le détail des commandes de configuration (verrouillage de fréquence à 4.0 GHz, isolation des cœurs) est documenté dans le fichier `scripts/README.md` et appliqué via les scripts `benchmark_affinity.sh` et `benchmark_large.sh`. Un exemple de script est également donné en Annexe B. Ce protocole s’inspire des bonnes pratiques de notre ancien cours, adaptées ici à l’architecture AMD Ryzen. Les mesures de temps sont réalisées par le programme C++ `tests/test_benchmark_large.cpp`, dédié au benchmarking du noyau de calcul, qui intègre une horloge haute résolution (`std::chrono`) pour exclure les surcoûts liés au système d’exploitation. Le protocole inclut systématiquement un préchauffage (*warm-up*) de 5 itérations pour stabiliser les caches (L1/L2) et le *branch predictor* avant le début des mesures.

Maîtrise de l'aléatoire Au-delà du contrôle matériel, la rigueur expérimentale est assurée par le verrouillage des sources d'aléatoire logiciel. L'initialisation des poids et le mélange des données (*data shuffling*) dépendent d'une graine unique fixée via la ligne de commande (`-seed`). Cela permet une reproduction bit-à-bit des courbes d'apprentissage et des métriques finales, condition indispensable pour comparer objectivement l'impact des différentes méthodes d'initialisation ou des hyperparamètres.

5.2 Analyse des performances de calcul et d'entraînement

Cette section détaille les résultats obtenus lors des campagnes de test, en analysant l'impact du parallélisme, de l'affinité et des optimisations bas niveau sur les performances. Il convient de préciser que ces micro-benchmarks portent exclusivement sur l'opération de multiplication matricielle (`matmul`), cœur intensif du calcul, et non sur le processus d'entraînement complet du réseau de neurones.

5.2.1 Passage à l'échelle (Thread Scaling)

L'évaluation du passage à l'échelle (*scaling*) de l'implémentation OpenMP a été réalisée en faisant varier le nombre de threads de 1 à 16, sur une architecture disposant de 8 cœurs physiques.

Une attention particulière a été portée à la précision des mesures pour les petites instances. Les premiers essais, pilotés par des scripts externes, présentaient une variabilité excessive. Pour y remédier, la boucle de mesure a été intégrée directement **au sein du programme C++** (`test_benchmark_large.cpp`), isolant ainsi le noyau de calcul des surcoûts liés au lancement de processus. Le protocole final adopte une méthodologie rigoureuse : un nombre d'itérations adaptatif (de 100 pour $N = 64$ à 5 pour $N = 2048$), précédé systématiquement de 5 exécutions de chauffe (*warm-up*). L'application d'une moyenne tronquée (rejet des 10% de valeurs hautes et 10% de valeurs basses) a permis de réduire l'écart type à environ $2 \mu s$ pour les configurations stables (≤ 8 threads), garantissant la fiabilité des résultats présentés ci-après.

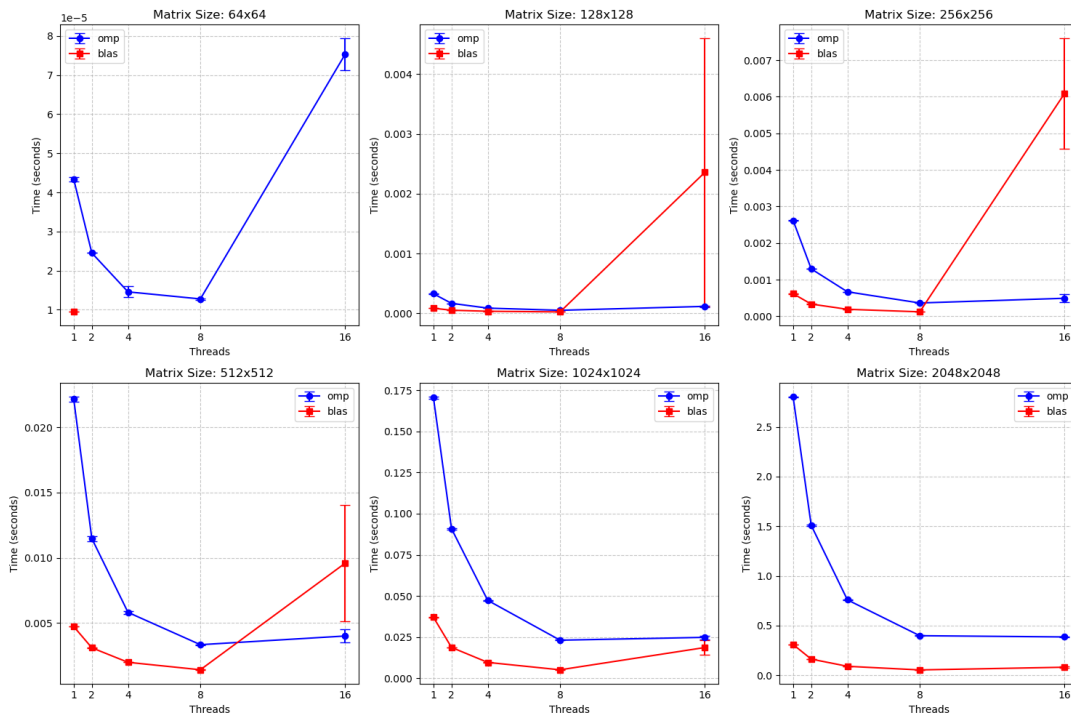


Figure 3: Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices.

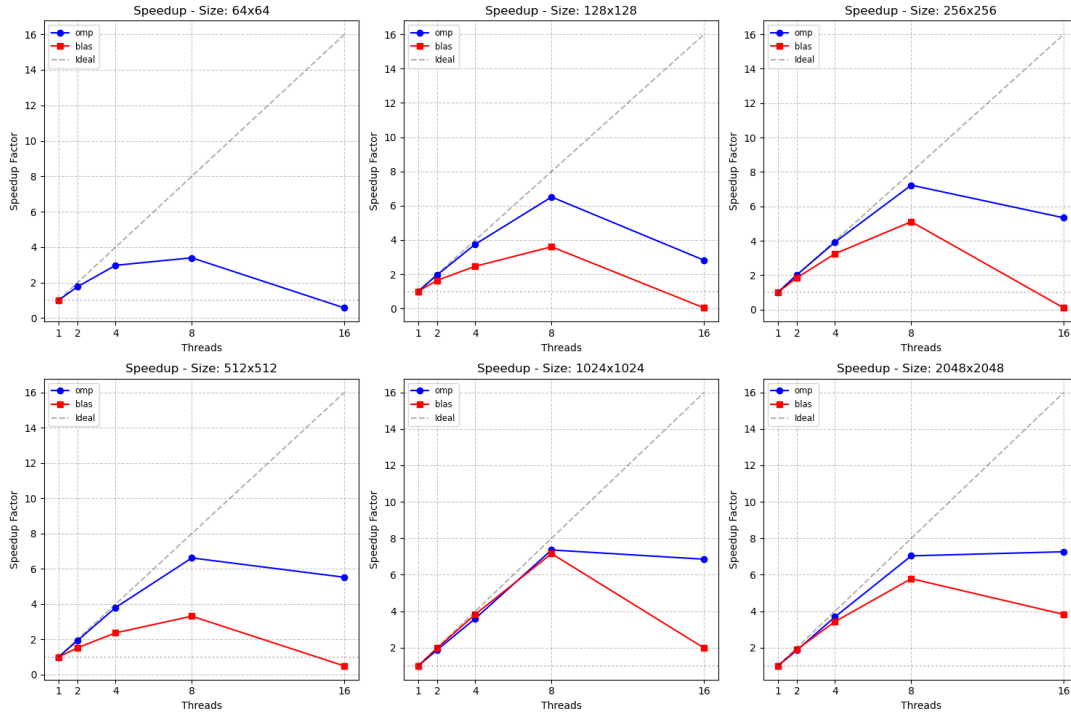


Figure 4: Speedup (accélération) en fonction du nombre de threads.

Analyse des petites matrices ($N = 64 \times 64$). La parallélisation OpenMP apporte un gain conforme aux attentes entre 2 et 8 threads, mais devient contre-productive au-delà de 8 threads (au-delà du nombre de cœurs physiques). À cette échelle, la granularité du calcul est trop fine : la charge utile par thread devient comparable, voire inférieure, au surcoût de gestion (création, synchronisation et contention des ressources), ce qui annule les gains, voire dégrade les performances.

À l'inverse, BLAS reste monothread sur des matrices 64×64 , la charge de calcul n'atteignant pas son seuil interne de déclenchement du parallélisme. Ainsi, pour cette échelle, les configurations les plus efficaces consistent soit à utiliser BLAS, soit à recourir à OpenMP en limitant le nombre de threads au nombre de cœurs physiques (8 threads dans notre cas).

Matrices de taille intermédiaire (128×128 à 1024×1024). Pour les matrices de taille intermédiaire, OpenMP présente une accélération nette lorsque le nombre de threads augmente jusqu'au nombre de cœurs physiques disponibles. Au-delà de ce seuil, les gains se tassent, voire deviennent négatifs, sous l'effet de la saturation des ressources partagées et de l'augmentation des coûts de synchronisation.

BLAS offre de meilleures performances globales grâce à ses optimisations internes (vectorisation et découpage en blocs). Cependant, lorsque le nombre de threads dépasse le nombre de cœurs physiques, ses performances se dégradent et deviennent instables, principalement en raison de la contention des ressources et des changements de contexte (*context switching*) sur un même cœur.

Grandes matrices (2048×2048). Pour les matrices de grande taille, les deux approches bénéficient d'une parallélisation efficace jusqu'à environ 8 threads. Toutefois, l'augmentation du nombre de threads au-delà de ce point n'entraîne plus de gain significatif, le calcul devenant principalement limité par la bande passante mémoire. Dans ce contexte, BLAS reste la solution la plus performante et la plus stable.

Ainsi, l'efficacité du parallélisme dépend non seulement de la taille du problème, mais aussi d'une adéquation stricte entre nombre de threads et ressources matérielles disponibles.

5.2.2 Hiérarchie des performances et Speedup

La comparaison globale des implémentations sur grandes matrices met en évidence cinq paliers de performance (Figure 5) :

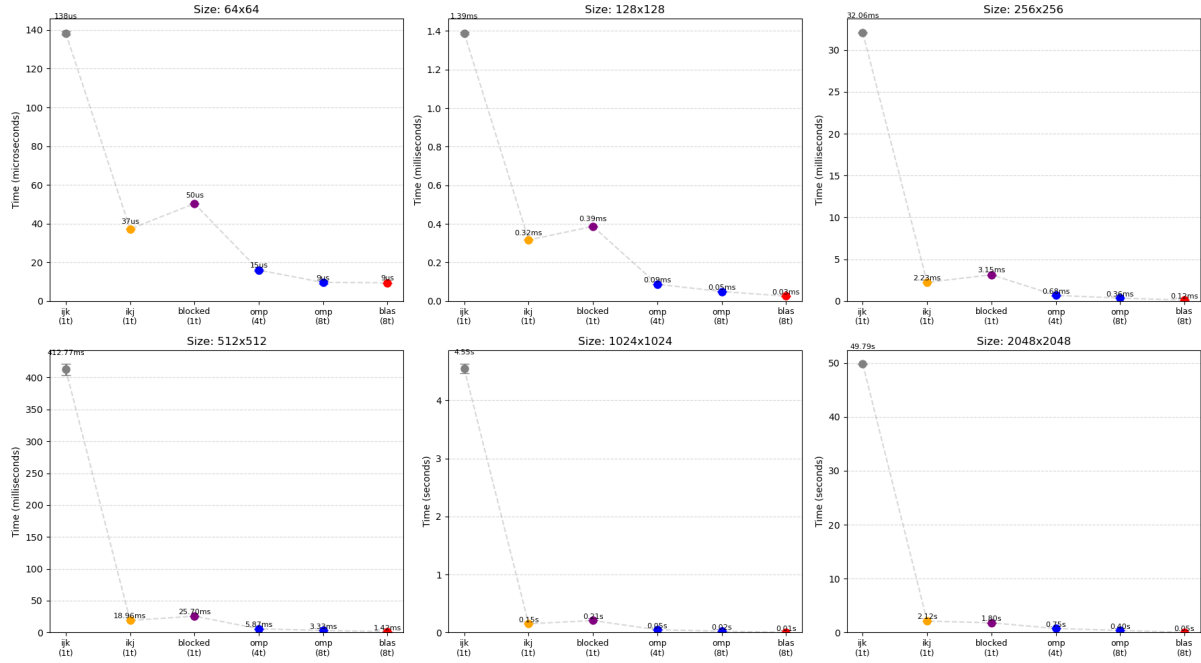


Figure 5: Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS).

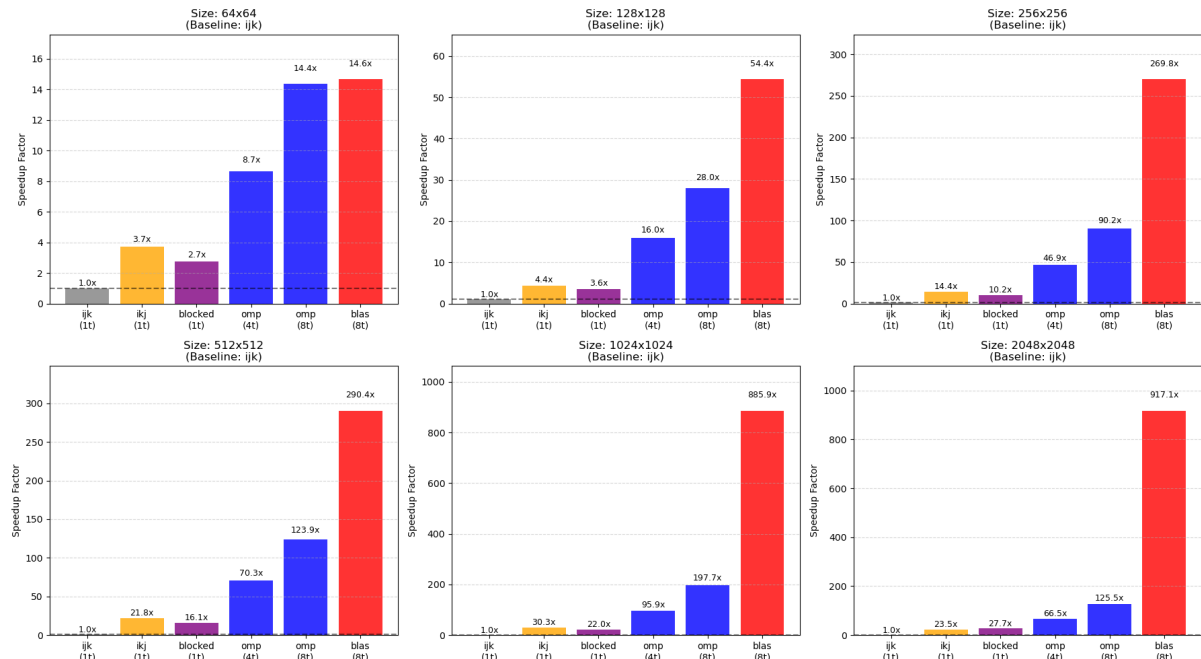


Figure 6: Speedup relatif des différentes optimisations.

1. **Naïf (ijk)** : L'ordre des boucles induit un parcours mémoire très défavorable, entraînant

de nombreuses fautes de cache. Le temps d'exécution devient prohibitif pour les grandes tailles, atteignant plusieurs secondes pour $N = 2048$.

2. **Réordonné (*ikj*)** : La permutation des boucles améliore la localité spatiale des accès mémoire et permet un premier gain significatif par rapport à l'implémentation naïve, en particulier pour les tailles intermédiaires et grandes.
3. **Bloqué (blocked)** : Cette optimisation apporte un gain mesurable par rapport à la version réordonnée, en améliorant la réutilisation des données dans les caches.
4. **OpenMP** : La parallélisation sur 8 cœurs permet d'exploiter le parallélisme matériel et apporte un gain supplémentaire, bien que celui-ci reste contraint par le nombre de cœurs physiques et par la hiérarchie mémoire.
5. **BLAS (OpenBLAS)** : Grâce à la combinaison d'optimisations avancées — blocage hiérarchique, vectorisation (AVX2/FMA) et micro-noyaux optimisés — BLAS surpasse largement les implémentations manuelles, avec un speedup pouvant atteindre plusieurs centaines de fois par rapport à la version naïve pour les plus grandes tailles.

5.2.3 Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet

Afin de comprendre l'impact de ces optimisations sur le temps total d'apprentissage, nous avons profilé une époque complète d'entraînement (forward + backward) avec `gprof`.

Architecture : MLP $784 \rightarrow 128 \rightarrow 10$ avec ReLU

Taille du mini-batch : 64

Données : MNIST Train (60 000 images)

Avant optimisation (Kernel Naïf) : L'application est totalement limitée par le calcul (*compute-bound*). La fonction `Matrix::matmul` consomme **environ 96%** du temps total (Tableau 3).

Table 3: Profil d'exécution (gprof) - Version Naïve : `Matrix::matmul` domine (96.4%)

% Time	Self (s)	Calls	Function Name
96.42	11.84	6256	<code>Matrix::matmul</code>
1.06	0.13	1252	<code>DataLoader::nextBatch</code>
0.73	0.09	3752	<code>Matrix::transpose</code>
0.41	0.05	12216	<code>Matrix::Matrix (constructor)</code>
0.24	0.03	95M	<code>Matrix::operator() const</code>
0.24	0.03	74M	<code>Matrix::operator() (mut)</code>
0.24	0.03	9380	<code>Node::addGrad</code>
0.24	0.03	1	<code>MNISTDataset::load</code>
0.16	0.02	-	<code>_init</code>
0.08	0.01	2504	<code>MathOps::add</code>

Après optimisation (BLAS) : Le temps passé dans `Matrix::matmul` s'effondre à moins de **1%** ($\approx 0.5s$). Le goulot d'étranglement se déplace alors vers :

- Le chargement des données (`MNISTDataset::load`) : $\approx 43\%$ du temps. Ce coût élevé s'explique par l'allocation et l'initialisation à zéro (inhérente au constructeur `std::vector`) d'un volume important de mémoire (376 Mo), suivies d'une copie avec conversion de type (`uint8_t` vers `double`) pour chaque pixel.
- Les allocations mémoire (`Matrix::Matrix`) : $\approx 29\%$ du temps.

Table 4: Profil d'exécution (gprof) - Version BLAS : `matmul` négligeable, overhead mémoire dominant

% Time	Self (s)	Calls	Function Name
42.86	0.03	1	MNISTDataset::load
28.57	0.02	12216	Matrix::Matrix (init double)
28.57	0.02	938	SGDOptimizer::step
0.00	0.00	6256	Matrix::matmul (optimisé par BLAS)
0.00	0.00	95M	Matrix::operator() const
0.00	0.00	74M	Matrix::operator() (mut)
0.00	0.00	40k	std::mersenne_twister
0.00	0.00	18k	Matrix::Matrix (copy)
0.00	0.00	10k	std::_Hashtable...
0.00	0.00	10k	Node::Node

Conséquence (Loi d’Amdahl) : Bien que le noyau de calcul ait été accéléré de plusieurs centaines de fois au niveau micro-benchmark, l’accélération globale de l’application est limitée par les parties non optimisables, telles que le chargement des données et les allocations mémoire. Le temps total d’une époque, mesuré via le script dédié `benchmark_e2e.sh` (compilation Release sans surcoût de profilage, moyenne sur 5 exécutions), passe ainsi de 13.27s à 1.74s, soit un **speedup end-to-end de 7.6x**. Ce résultat illustre clairement la loi d’Amdahl [5] : une fois le calcul optimisé, les gains supplémentaires nécessitent d’agir sur la gestion mémoire et les entrées/sorties.

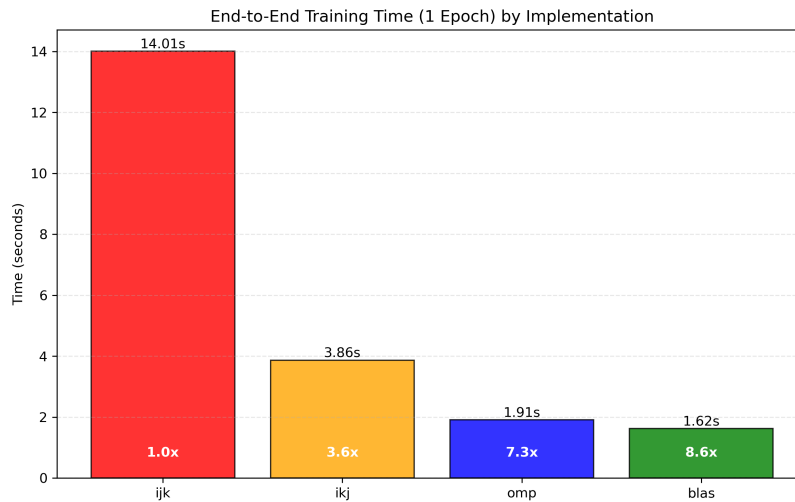


Figure 7: Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS.

5.3 Convergence de l'entraînement

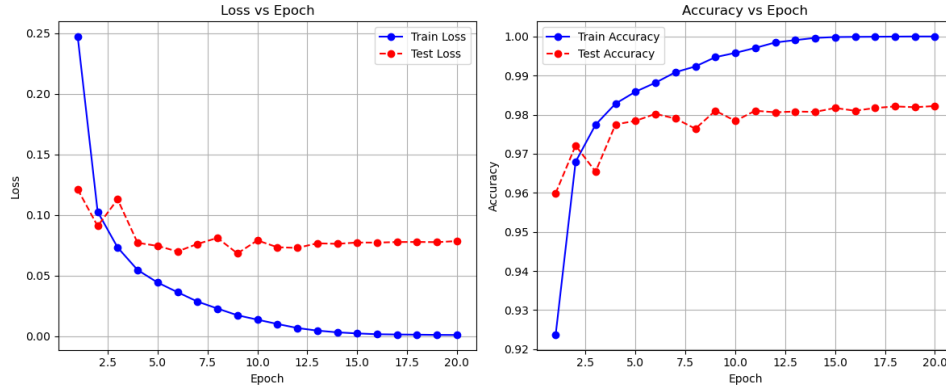
5.3.1 Premiers tests de convergence

Afin de valider la correction du modèle et de l'algorithme d'optimisation, nous présentons la courbe d'apprentissage sur MNIST. L'ensemble des scripts et commandes utilisés pour reproduire ces expériences est disponible en Annexe C.

Les paramètres utilisés pour ce premier test sont :

Paramètre	Valeur
Taux d'apprentissage (LR)	0.01
Taille de batch (B)	64
Taille cachée (H)	128
Fonction d'activation	ReLU
Initialisation	He
Époques	20

Table 5: Configuration standard pour le test de convergence.

Figure 8: Courbe de *loss* et de précision (*accuracy*) sur les données d'entraînement (60 000 images) et de validation (10 000 images).

Commentaire sur la convergence (Figure 8) : Cette figure présente deux graphiques : la *loss* (à gauche) et la précision (à droite).

- **Courbe bleue (*train*) :** performance sur les données d'entraînement. Elle reflète la capacité du modèle à s'ajuster aux données vues.
- **Courbe rouge (*validation*) :** performance sur les données de validation, jamais vues pendant l'entraînement.

Interprétation : La courbe rouge suit la bleue mais reste légèrement en retrait (*loss* plus élevée, précision plus faible). C'est attendu : un modèle obtient généralement de meilleures performances sur les données d'entraînement que sur des données inédites. Si l'écart devenait très important, cela indiquerait un sur-apprentissage (*overfitting*) ; ici, il reste modéré, suggérant une généralisation correcte.

5.3.2 Impact du Taux d'Apprentissage (Learning Rate)

Nous avons évalué l'impact du taux d'apprentissage sur la convergence du modèle en testant les valeurs $\{0.025, 0.01, 0.0075, 0.005, 0.001\}$. Les autres hyperparamètres ont été fixés : batch $B = 64$, couche cachée $H = 128$, activation ReLU et initialisation He. Pour assurer la robustesse statistique, chaque configuration a été exécutée trois fois avec des graines différentes.

La Figure 9 présente les résultats obtenus.

Learning Rate Experiment Results (10 Epochs)

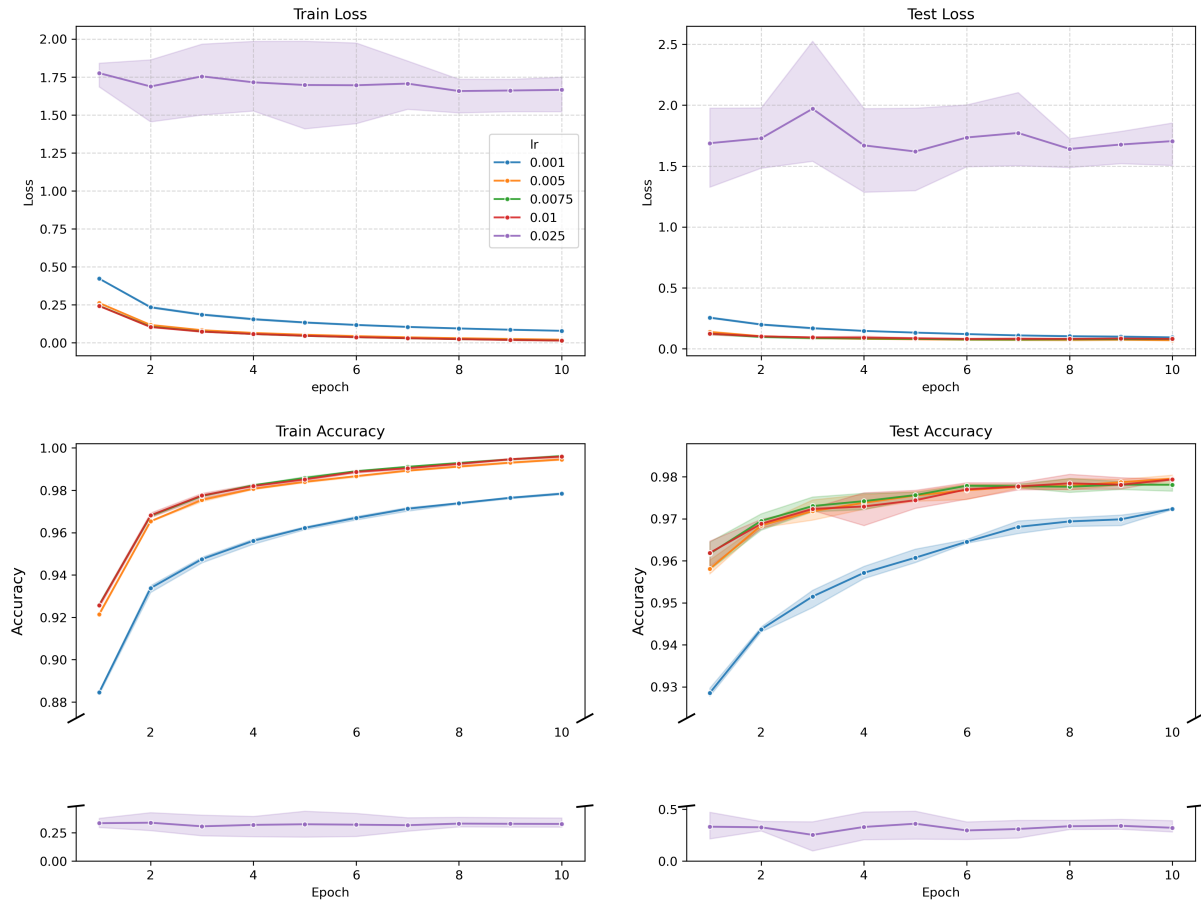


Figure 9: Impact du *learning rate* sur la convergence (10 époques, 3 graines). Les axes de précision utilisent une échelle discontinue pour visualiser simultanément les hautes précisions ($> 90\%$) et les échecs de convergence ($< 40\%$).

Analyse des résultats :

- **Taux trop faible** ($LR = 0.001$) : Le modèle converge mais trop lentement. Après 10 époques, la précision de validation plafonne à $\sim 97.2\%$, inférieure à celle obtenue avec des taux plus élevés. Les mises à jour des poids sont trop petites pour atteindre l'optimum rapidement.
- **Zone optimale** ($LR \in [0.005, 0.01]$) : Ces taux offrent le meilleur compromis. La convergence est rapide et stable, atteignant une précision de $\sim 97.9\%$ à $\sim 98.0\%$. $LR = 0.01$ est retenu pour sa rapidité.
- **Taux trop élevé** ($LR = 0.025$) : On observe une instabilité critique (divergence). Les pas de gradient sont trop grands, faisant "sauter" l'optimiseur par-dessus les minima de la *loss*. La précision s'effondre à $\sim 32.1\%$ avec une variance élevée ($\sigma \approx 0.06$), rendant l'entraînement inefficace.

Ces résultats expérimentaux confirment directement les intuitions théoriques présentées en section 3.3.2 concernant les risques de divergence (taux trop grand) et la lenteur de convergence (taux trop faible).

Nous retenons donc $LR = 0.01$ comme valeur optimale pour la suite.

5.3.3 Impact de la Taille de Batch (Batch Size)

En fixant $LR = 0.01$, nous avons étudié l'impact de la taille de batch $\{16, 32, 64, 128, 256\}$ sur la stabilité et la performance.

La Figure 10 illustre ces résultats.

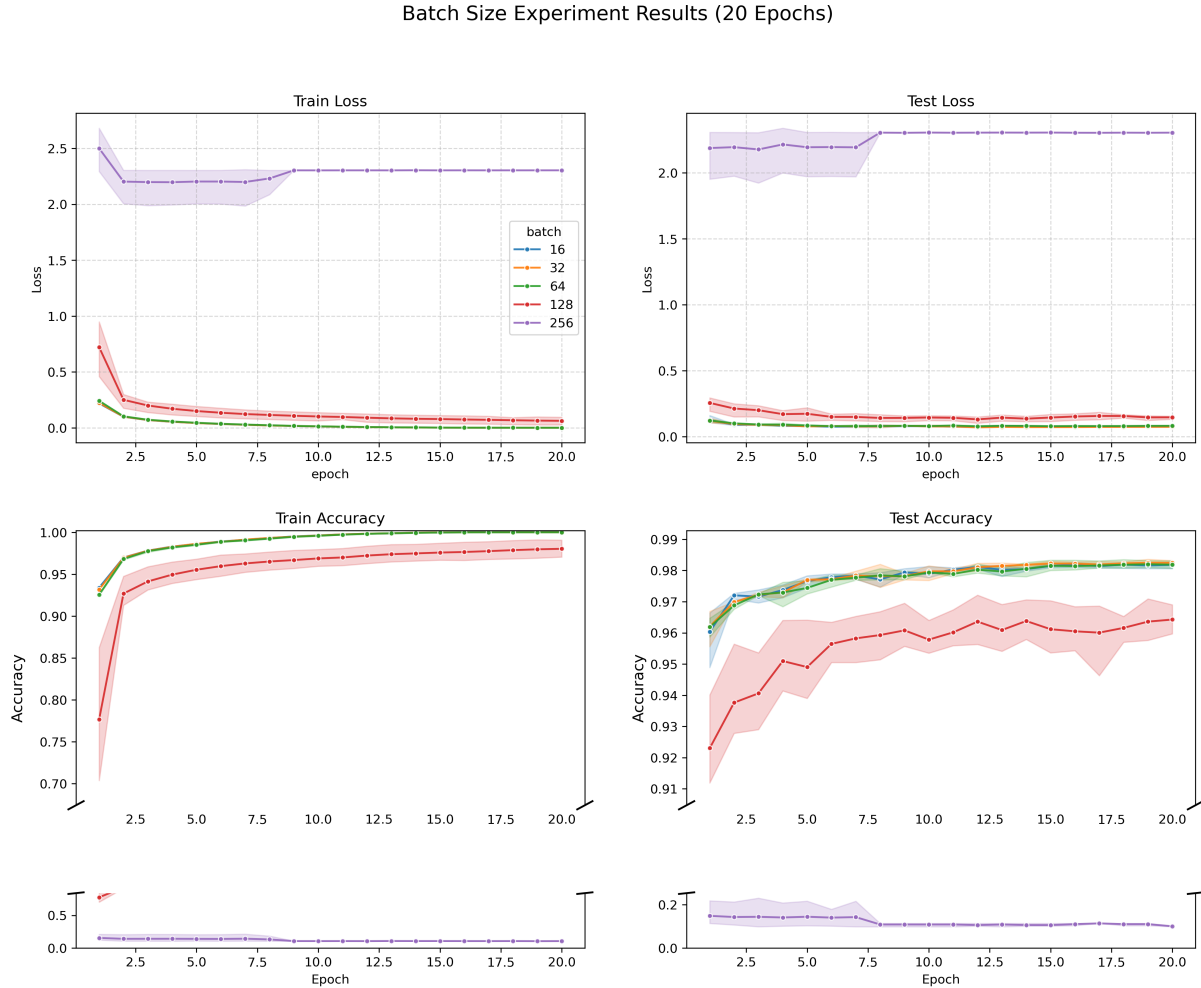


Figure 10: Impact de la Taille de Batch (20 époques, $LR = 0.01$, 3 graines). Les petits batchs (16, 32) convergent mieux mais sont plus lents à exécuter. Le batch 256 diverge avec ce LR fixe.

Analyse :

- **Petits Batchs (16, 32) :** Offrent la meilleure précision finale ($\sim 98.2\%$) grâce à l'introduction de bruit dans les gradients qui aide à la généralisation. Aucune différence significative n'est observée entre 16 et 32.
- **Batch Moyen (64) :** Un compromis idéal offrant une précision comparable ($\sim 98.2\%$) avec une meilleure efficacité de calcul (plus grande parallélisation).
- **Grands Batchs (128, 256) :** La performance se dégrade avec $B = 128$, et le modèle diverge complètement avec $B = 256$. Pour stabiliser $B = 256$, il faudrait probablement ajuster le LR (*linear scaling rule*) ou utiliser un *warm-up*, mais avec $LR = 0.01$ fixe, cela échoue.

Note : Les grands batchs sont plus rapides à exécuter par époque, car les opérations matricielles plus larges exploitent mieux le parallélisme matériel (CPU/GPU) et la vectorisation. Les petits batchs génèrent davantage d'appels noyau (surcoût) et n'utilisent qu'une partie des unités de calcul, ce qui les rend plus lents malgré leur meilleure stabilité d'optimisation.

5.3.4 Impact de la Taille de la Couche Cachée (Hidden Size)

Enfin, en fixant $LR = 0.01$ et un batch $B = 64$, nous avons évalué l'impact du nombre de neurones dans la couche cachée $\{64, 128, 256\}$ sur la capacité du modèle.

La Figure 11 illustre ces résultats.

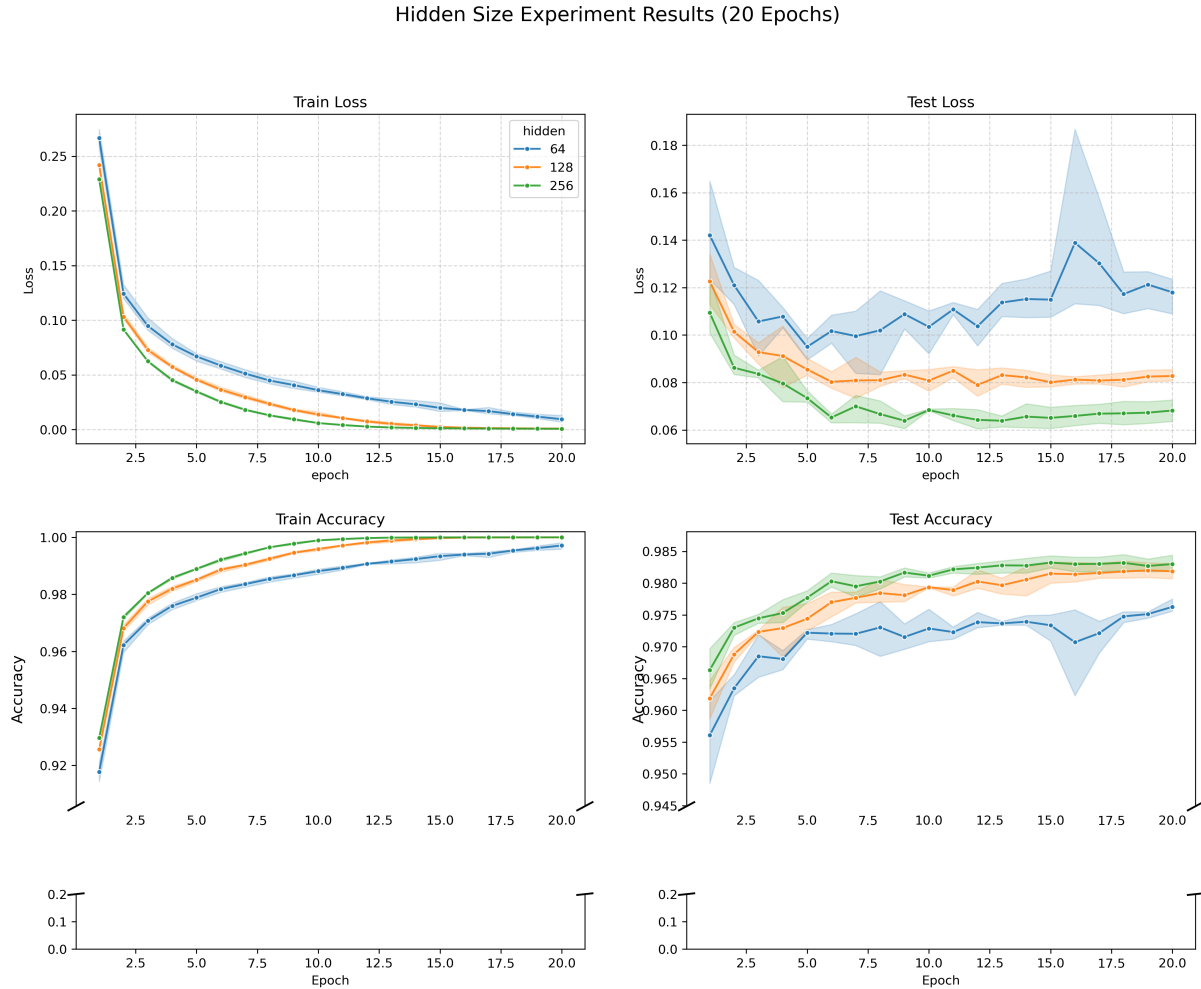


Figure 11: Impact de la Taille de la Couche Cachée (20 époques, $LR = 0.01$, $B = 64$, 3 graines). L'augmentation de la taille améliore la précision, mais avec des rendements décroissants au-delà de 128 neurones.

Analyse :

- **Capacité Limitée ($H = 64$) :** Le modèle sous-apprend légèrement, atteignant une précision de $\sim 97.6\%$. Cela suggère que 64 neurones ne suffisent pas tout à fait à capturer toute la complexité de MNIST avec cette architecture simple.
- **Compromis Optimal ($H = 128$) :** Avec une précision de $\sim 98.2\%$, cette configuration offre un gain significatif par rapport à $H = 64$ sans augmenter excessivement le coût de calcul.
- **Rendements Décroissants ($H = 256$) :** Bien que cette configuration offre la meilleure précision ($\sim 98.3\%$), le gain de 0.1% est marginal par rapport au doublement du coût de calcul de la couche cachée. Pour cette tâche, $H = 128$ est suffisant.

5.3.5 Impact de la Fonction d'Activation et de l'Initialisation

Nous avons comparé les fonctions d'activation ReLU, Sigmoid et Tanh (avec $LR = 0.01$, $Batch = 64$, $H = 128$). Pour garantir une comparaison équitable, nous avons utilisé l'initialisation de **He** pour ReLU et de **Xavier** pour Sigmoid/Tanh. De plus, nous avons inclus une initialisation dite "Manuelle" (*Manual Init*), correspondant à une distribution uniforme simple $\mathcal{U} \sim [-0.1, 0.1]$. Cette méthode naïve sert de point de référence (baseline) pour évaluer l'efficacité des méthodes modernes.

La Figure 12 présente les courbes de convergence.

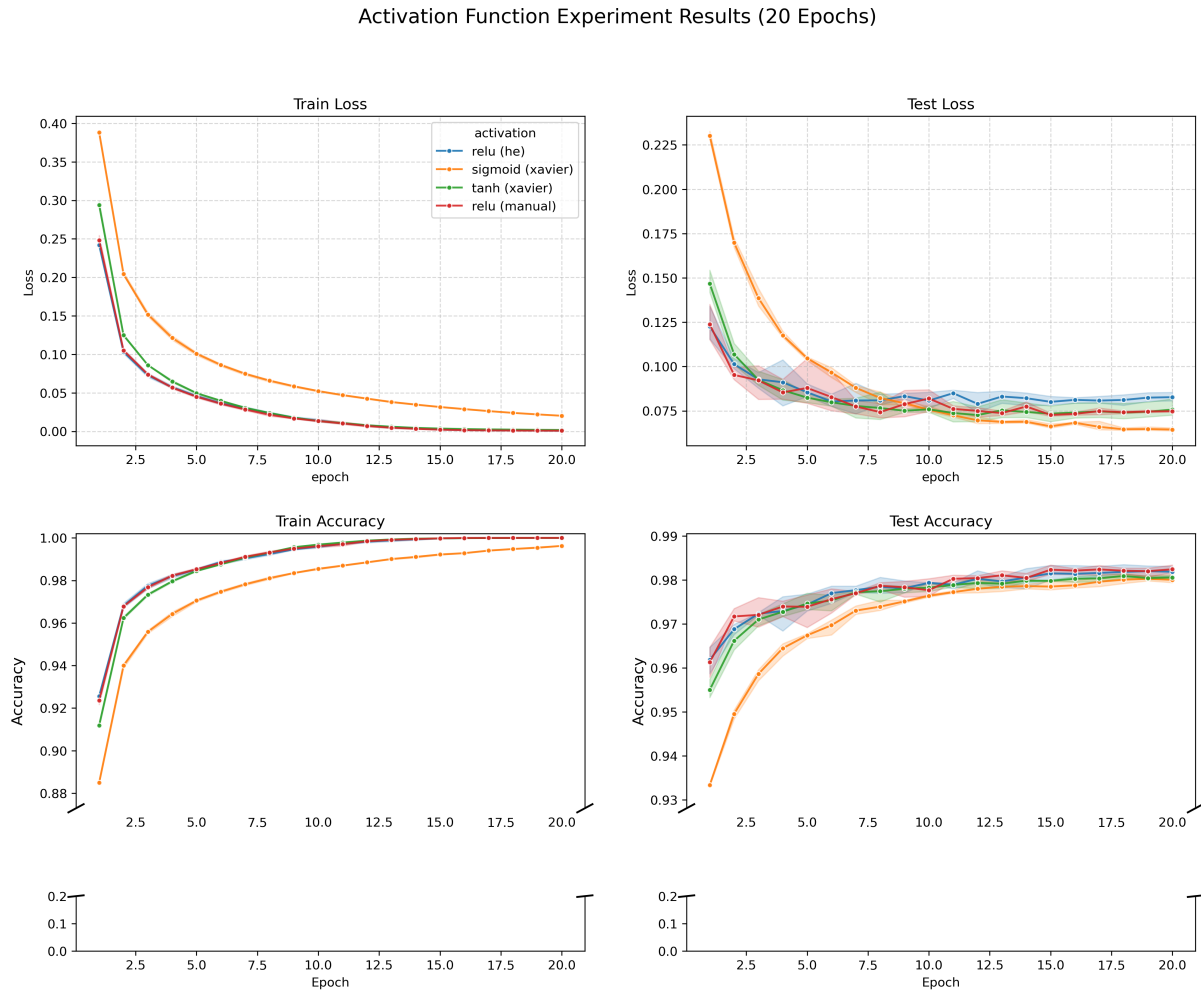


Figure 12: Comparaison des Fonctions d'Activation et Initialisation. ReLU (He) et ReLU (Manual) offrent les meilleures performances. Sigmoid et Tanh (avec Xavier) restent compétitifs grâce à une initialisation adaptée.

Analyse des performances (précision de validation moyenne) :

- **ReLU (Manual Init)** [$\sim 98.24\%$] : Notre initialisation manuelle $[-0.1, 0.1]$ a obtenu une performance comparable à l'initialisation de He. Cela s'explique par le fait que pour $N_{in} = 784$, l'écart-type de cette distribution uniforme (~ 0.057) est très proche de l'optimum théorique de He (~ 0.05).
- **ReLU (He Init)** [$\sim 98.19\%$] : La performance est quasi-identique à l'initialisation manuelle, confirmant la robustesse de ReLU sur ce problème.
- **Tanh (Xavier)** [$\sim 98.06\%$] et **Sigmoid (Xavier)** [$\sim 98.00\%$] : Bien que légèrement inférieures à ReLU, ces activations "historiques" performant bien ici car l'initialisation de

Xavier empêche efficacement la saturation des gradients au début de l’entraînement. Sans cette initialisation spécifique, elles auraient probablement échoué à converger aussi vite.

Synthèse théorique (Quand utiliser quoi ?) :

- **ReLU** : Le standard actuel pour les **couches cachées** des réseaux profonds (CNN, MLP). Elle évite le problème de disparition du gradient (Vanishing Gradient) et est très rapide à calculer.
- **Sigmoid** : Principalement utilisée pour la **couche de sortie** dans les problèmes de classification binaire (probabilité $p \in [0,1]$). À éviter dans les couches cachées profondes car elle sature (gradient $\rightarrow 0$).
- **Tanh** : Préférée à la Sigmoid dans les couches cachées si l’on souhaite des sorties centrées sur 0 (utile pour les RNNs), mais souffre aussi de saturation.

Conclusion finale des expériences : La configuration optimale retenue est : $LR = 0.01$, $B = 64$, $H = 128$, **activation ReLU**, **init. He/Manual**.

5.3.6 Impact de l’Affinité sur les charges d’entrée (Input Layer)

Les résultats présentés dans cette section analysent l’impact de l’affinité sur les opérations typiques de la couche d’entrée avec une taille de batch $B = 64$ (soit une multiplication $64 \times 784 \times 128$, charge de travail comparable à une matrice carrée intermédiaire). Le tableau ci-dessous détaille les résultats pour 2 et 8 threads.

Table 6: Impact de l’affinité sur les performances pour une multiplication matricielle $64 \times 784 \times 128$.

Threads	Affinité	Temps OMP (μs)	Temps BLAS (μs)	Ratio OMP/BLAS
2	default	480	132	3.6x
2	close	474	128	3.7x
2	spread	487	159	3.1x
8	default	149	68	2.2x
8	close	152	45	3.4x
8	spread	236	75	3.1x

L’analyse de ces données pour une multiplication $64 \times 784 \times 128$ révèle :

- **Impact de l’Affinité sur BLAS** : L’utilisation de l’affinité **close** est critique. À 8 threads, elle permet de descendre à $45\mu s$, contre $68\mu s$ en configuration par défaut (**default**), soit un gain de **33%**. La configuration **spread** est la moins performante ($75\mu s$), dispersant les threads loin des caches partagés.
- **Impact sur OpenMP** : Notre implémentation naïve réagit différemment. À 8 threads, le mode **default** ($149\mu s$) et **close** ($152\mu s$) sont équivalents, tandis que **spread** dégrade massivement les performances ($236\mu s$). Cela indique que le système d’exploitation place par défaut les threads de manière raisonnablement compacte, mais que forcer la dispersion nuit gravement à la cohérence de cache pour notre algorithme.

Conclusion : Pour les opérations de la couche d’entrée ($64 \times 784 \times 128$), l’utilisation de bibliothèques optimisées comme BLAS est impérative. Cependant, ces bibliothèques sont très sensibles à la localité : pour obtenir le gain de performance maximal (facteur $\times 4$ face à OpenMP), il est crucial de configurer l’affinité des threads (mode **close**) afin de minimiser les latences cache.

5.4 Synthèse des Résultats Expérimentaux

L'ensemble de ces expérimentations met en lumière la dualité du défi posé par les réseaux de neurones : un défi **algorithmique** (trouver les bons hyperparamètres pour la convergence) et un défi **computationnel** (exécuter ces calculs efficacement).

Aspect algorithmique : La recherche par grille a permis d'isoler une configuration robuste (**LR=0.01, Batch=64, H=128, ReLU**) capable d'atteindre $\sim 98.2\%$ de précision sur MNIST, validant notre implémentation de la rétropropagation. **Aspect performance :** L'optimisation bas niveau s'avère critique. L'utilisation de bibliothèques vectorisées (BLAS) avec une affinité CPU contrôlée (`close`) offre des gains de performance considérables sur le noyau de calcul ; dans nos mesures, la configuration la plus efficace correspond à **8 threads** avec `OMP_PROC_BIND=close`. Cependant, l'accélération globale de l'apprentissage reste bornée par les coûts de gestion mémoire et de chargement de données (loi d'Amdahl), soulignant l'importance d'une optimisation holistique du système.

6 Déroulé du Projet

L'état de l'art a été rédigé conjointement par Jianye, Hao et Xiang, afin de présenter le contexte théorique du projet et les approches existantes en matière de réseaux de neurones et de classification sur MNIST. L'équipe a ensuite organisé son travail en plusieurs phases successives, chacune portée par un ou plusieurs membres selon leurs affinités techniques.

La phase de conception a été initiée par Jianye, qui a rédigé la documentation préliminaire et produit la première version du diagramme UML, permettant de clarifier l'architecture générale du projet. La mise en place du premier prototype fonctionnel du MLP a ensuite été assurée par Abdenour, qui a développé les modules fondamentaux liés aux tenseurs, aux couches linéaires et aux fonctions d'activation.

La réalisation du moteur de différenciation automatique a été répartie entre Hao et Xiang, chacun prenant en charge une partie des opérations nécessaires au calcul du graphe et des gradients. Par la suite, Jianye a intégré l'ensemble de ces composants, généré une seconde version du diagramme UML pour refléter les ajustements apportés, et affiné la conception en fonction des problèmes rencontrés.

Le développement du mécanisme d'entraînement a été amorcé par Xiang, tandis que Jianye et Hao ont poursuivi respectivement l'implémentation de l'optimiseur et des fonctions de *loss*. L'intégration du jeu de données MNIST et la validation du pipeline complet ont été assurées par Jianye.

Jianye a également mené des tests de performance et des analyses de profilage afin d'optimiser l'ensemble du système, ainsi qu'une recherche expérimentale visant à identifier une configuration robuste des hyperparamètres.

Enfin, la rédaction du rapport a été réalisée collectivement.

7 Conclusion & perspectives

Ce projet a permis de démystifier le fonctionnement interne des réseaux de neurones profonds en implémentant, à partir de zéro (*from scratch*), un moteur complet d'apprentissage en C++. L'architecture retenue, basée sur un graphe de calcul dynamique (DAG) et la différenciation automatique, a démontré la flexibilité du C++ moderne (pointeurs intelligents, polymorphisme) pour modéliser des concepts mathématiques abstraits tout en garantissant une gestion rigoureuse de la mémoire.

Sur le plan de la performance, l'analyse expérimentale a mis en évidence l'impact critique des optimisations bas niveau. Si le noyau de calcul (`matmul`) a bénéficié d'une accélération spectaculaire de plus de trois ordres de grandeur ($1200\times$) grâce à BLAS et OpenMP, l'accélération

globale de l'application est restée plus modeste ($7.6\times$). Ce contraste illustre la loi d'Amdahl : une fois le calcul matriciel optimisé, le goulot d'étranglement se déplace inévitablement vers la gestion des données (*I/O*) et les allocations mémoire.

Limites et menaces à la validité : Les conclusions de performance dépendent d'un protocole de mesure contrôlé (verrouillage de fréquence, affinité CPU) et peuvent varier sur d'autres architectures, avec d'autres implémentations BLAS, ou sous une charge système différente. Par ailleurs, l'analyse se concentre principalement sur le noyau `matmul` dense et ne couvre pas nécessairement l'ensemble des coûts d'un entraînement complet (pipeline de données, allocations, et surcoûts d'orchestration). Enfin, le choix d'un MLP sur MNIST limite l'extrapolation à des architectures plus modernes (CNN) et à des jeux de données plus complexes.

Perspectives : Plusieurs axes d'amélioration pourraient enrichir ce projet :

- **Support GPU (CUDA) :** Déporter les calculs matriciels sur GPU pour accélérer massivement l'entraînement, en réduisant les transferts hôte-périphérique.
- **Vectorisation Explicite (SIMD) :** Remplacer l'auto-vectorisation du compilateur par des intrinsèques (AVX-512) ou des micro-noyaux optimisés pour exploiter pleinement les unités de calcul du CPU.
- **Optimisation Mémoire :** Implémenter un *memory pool* ou une arène d'allocation pour supprimer le surcoût de `malloc/free` identifié lors du profilage.
- **Parallélisme de Modèles :** Entraîner plusieurs instances du réseau en parallèle (par exemple pour l'exploration d'hyperparamètres), en complément du parallélisme de données déjà exploité par les *mini-batches*.
- **Extension aux CNN :** Intégrer des couches de convolution pour traiter des jeux de données plus complexes, une évolution naturelle envisagée pour la suite du cursus (S2).
- **Parallélisme Distribué (MPI) :** Étendre l'entraînement à plusieurs nœuds de calcul en utilisant MPI pour paralléliser le traitement des *mini-batches* (*data parallelism*).

8 Références

References

- [1] Ian Goodfellow, Yoshua Bengio, et Aaron Courville. *Deep Learning*. MIT Press, 2016. 8, 10
- [2] Yann LeCun, Léon Bottou, Yoshua Bengio, et Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 5, 6, 9
- [3] <https://m1-chps.github.io/ghpc/UVSQ/lecture5>, 2025. Consulté le 17/12/2025. 4, 7
- [4] David E. Rumelhart, Geoffrey E. Hinton, et Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. 7
- [5] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967. 21
- [6] OpenMP Architecture Review Board. *OpenMP Application Program Interface Version 5.0*. 2018. 13
- [7] Jack J. Dongarra et al. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)*, 16(1):1–17, 1990. 13

9 Glossaire

Contexte et État de l’art

Autodifférentiation	Calcul automatique des gradients en appliquant la règle de la chaîne sur un graphe de calcul.
MLP	<i>Multi-Layer Perceptron</i> : réseau <i>feed-forward</i> (couches linéaires + activations).
MNIST	Jeu de données de chiffres manuscrits (28×28, 10 classes) utilisé pour la classification.
HPC	<i>High-Performance Computing</i> : techniques visant à maximiser le débit de calcul.

Backpropagation Propagation inverse des gradients depuis la *loss* vers les paramètres.

Architecture et Modèles (CNN)

CNN	<i>Convolutional Neural Network</i> : réseau utilisant des couches de convolution, adapté aux données spatiales (images).
------------	---

Implémentation : Tenseurs et Réseaux

Tensor/Matrice	Structure de données pour stocker des valeurs et effectuer des opérations numériques (ici, matrices denses contiguës).
RNN	<i>Recurrent Neural Network</i> : réseau récurrent, adapté aux données séquentielles.

Implémentation : Optimisation

SGD *Stochastic Gradient Descent* : optimisation par mise à jour des paramètres à partir du gradient estimé sur un *mini-batch*.

Implémentation : Calcul et Matériel

CPU *Central Processing Unit* : processeur généraliste exécutant le programme.

SIMD *Single Instruction, Multiple Data* : exécution vectorielle de la même opération sur plusieurs données.

BLAS *Basic Linear Algebra Subprograms* : interface standard pour l'algèbre linéaire (p. ex. produits matriciels).

OpenMP API de parallélisme à mémoire partagée pour C/C++ et Fortran.

DAG *Directed Acyclic Graph* : graphe orienté sans cycle, utilisé comme graphe de calcul.

Logits Scores réels produits par le modèle avant normalisation (p. ex. avant *softmax*).

One-hot Encodage d'une classe par un vecteur binaire de dimension K (un seul 1, le reste à 0).

Softmax Transformation d'un vecteur de scores en probabilités (classification multi-classes).

Epoch Passage complet sur le jeu d'entraînement.

Expérimentations : Configuration

Affinité CPU Restriction du placement des threads/processus à un sous-ensemble de cœurs pour réduire la variabilité.

Warm-up Itérations de préchauffage avant mesure (stabilisation caches et exécution).

Expérimentations : Résultats

Speedup Accélération : rapport entre un temps de référence et un temps optimisé (p. ex. T_1/T_p).

I/O *Input/Output* : lecture/écriture de données, pouvant devenir un goulot d'étranglement.

Perspectives (GPU)

CUDA Plateforme et modèle de programmation NVIDIA pour l'exécution sur GPU.

GPU *Graphics Processing Unit* : processeur massivement parallèle, efficace pour l'algèbre linéaire.

Autres concepts

Overfitting Sur-apprentissage : le modèle s'ajuste trop aux données d'entraînement et généralise moins bien.

A Exemple : graphe de calcul et autodiff

Un **graphe de calcul** représente une expression comme un enchaînement d'opérations élémentaires. Par exemple, pour

$$z = x \cdot w, \quad y = z + b, \quad L = y^2,$$

le graphe contient trois nœuds d'opération (multiplication, addition, carré) reliés par des dépendances ($x, w \rightarrow z \rightarrow y \rightarrow L$). Lors du **passage avant**, chaque nœud calcule puis *stocke* sa valeur (ici z et y), afin d'éviter de recalculer ces intermédiaires lors du backward. Lors du **passage arrière** (autodiff en mode inverse), chaque nœud utilise sa **dérivée locale** et le gradient amont pour mettre à jour ses parents :

$$\frac{\partial L}{\partial y} = 2y, \quad \frac{\partial y}{\partial z} = 1, \quad \frac{\partial y}{\partial b} = 1, \quad \frac{\partial z}{\partial x} = w, \quad \frac{\partial z}{\partial w} = x.$$

On obtient alors, par composition, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$, $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z}$, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$ et $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$. Dans notre implémentation, cette logique se traduit par des nœuds **Node** stockant une **value** (résultat du forward) et une fonction **backwardFn_** (dérivée locale + accumulation du gradient dans les parents). Enfin, la **rétropropagation** correspond au cas particulier où ce graphe de calcul est celui d'un réseau de neurones : c'est l'application de l'autodiff en mode inverse à une composition de couches.

B Protocole de Verrouillage des Fréquences

Afin de garantir la reproductibilité des mesures, le script suivant est exécuté avant chaque campagne de test pour fixer la fréquence du CPU à 4.0 GHz sur les cœurs physiques.

Listing 5: Commandes de configuration CPU (mode performance)

```
# 1. Passer le gouverneur en mode performance
sudo cpupower frequency-set -g performance

# 2. Verrouiller la fréquence min et max à 4.0 GHz
# (Adaptez la valeur selon votre CPU)
sudo cpupower frequency-set -u 4000MHz -d 4000MHz

# 3. Vérifier la fréquence
cpupower frequency-info | grep "current_CPU_frequency"
```

Une fois les mesures terminées, l'environnement est restauré :

Listing 6: Restauration de l'environnement

```
sudo cpupower frequency-set -d 421MHz -u 5386MHz
sudo cpupower frequency-set -g powersave
```

C Scripts d'Expérimentation

L'ensemble des expériences (Grid Search) a été automatisé via des scripts Bash invoquant l'exécutable **ppn_train** avec les hyperparamètres appropriés. Voici un exemple représentatif pour l'étude de la taille de couche cachée :

Listing 7: Exemple de script d'automatisation (exp_hidden_size.sh)

```
#!/bin/bash
# Configuration
EXE="./build/ppn_train"
EXP_DIR="output/experiments/hidden"
```



```
mkdir -p "$EXP_DIR"

# Hyperparameters to test
# Taille de couche cachée à tester
# On peut aussi varier le taux d'apprentissage, le batch size, activation
  function et initialisation.
HIDDEN_SIZES=(64 128 256)
SEEDS=(42 43 44)
EPOCHS=20

# Fixed parameters (Control Variates)
LR=0.01
BATCH=64
ACTIVATION="relu"
INIT="he"

for hidden in "${HIDDEN_SIZES[@]}; do
  for seed in "${SEEDS[@]}; do
    # Run command
    $EXE --learning_rate $LR \
        --batch_size $BATCH \
        --hidden_size $hidden \
        --activation $ACTIVATION \
        --init $INIT \
        --epochs $EPOCHS \
        --seed $seed \
    > "$EXP_DIR/hidden_${hidden}_seed${seed}.log"
  done
done
```